

FoodieGo

Advanced Food Delivery Microservices System

Cloud Computing Capstone Project

Level 4 — IS Department

Student ID	Student Name
2022170043	أحمد مصطفى عبدالراضي حسين
2022170595	كمال الدين سيد كمال عودة
2022170283	عمر محمد سيد احمد
2022170265	عمر احمد عباس عبدالعليم
2022170457	ميناء توفيق حمادة
2022170590	عمر محمد وفيق ذكي محمد العطار

Contents

1	Project Foundation	2
1.1	Mission and Scope	2
1.2	Core Architectural Features	2
2	System Architecture	3
2.1	Decomposition of Services	3
2.2	The Async Order Workflow	3
3	Technology Stack	4
3.1	Platform Components	4
3.2	Database Multi-Tenancy Strategy	4
4	Monitoring & Observability	5
4.1	The Monitoring Stack	5
4.2	Health Monitoring	5
5	Setup and Deployment	6
5.1	Environment Configuration	6
5.2	Kubernetes Pipeline	6
6	Closing Remarks	7

Chapter 1

Project Foundation

1.1 Mission and Scope

FoodieGo is a high-performance, full-stack food delivery application architected using modern cloud-native principles. This project serves as a comprehensive demonstration of a distributed system designed to solve the challenges of scalability, availability, and maintainability in a high-traffic environment.

The system empowers customers with a seamless ordering experience while providing restaurant owners and drivers with robust operational dashboards. Under the hood, it leverages a complex interplay of RESTful APIs, asynchronous message brokering, and container orchestration.

1.2 Core Architectural Features

- **Microservices Topology:** Discrete services for User, Restaurant, Order, and Payment domains.
- **Resilient Messaging:** RabbitMQ ensures that payment processing is decoupled and fault-tolerant.
- **Cloud-Native Orchestration:** Fully defined Kubernetes manifests for high-availability deployments.
- **Unified Monitoring:** Integrated **Prometheus** and **Grafana** stack to provide deep visibility into system performance and container health.

Chapter 2

System Architecture

2.1 Decomposition of Services

The platform is decomposed into four core microservices, each fulfilling a specific domain requirement:

Microservice Directory

User Service (3001): Manages identity, JWT-based authentication, and profile state.

Restaurant Service (3002): Handles the catalog of menu items and restaurant metadata.

Order Service (3003): Acts as the system's "brain," coordinating the complex lifecycle of an order.

Payment Service (3004): An async worker that processes financial transactions off a queue.

2.2 The Async Order Workflow

To ensure the user interface remains responsive, we employ an asynchronous pattern for payment processing:

1. **Client Request:** The React app hits the Order Service.
2. **Cross-Service Validation:** The Order Service verifies user identity and menu pricing via internal REST calls.
3. **State Persistence:** The order is written to PostgreSQL with an initial `PLACED` status.
4. **Message Dispatch:** A payload is sent to the RabbitMQ `payment_queue`.
5. **Background Processing:** The Payment Service picks up the message, completes the transaction, and logs it in the ledger.

Chapter 3

Technology Stack

3.1 Platform Components

Layer	Technology	Purpose
Backend	Node.js 20 / Express 4	Low-latency, non-blocking I/O.
Frontend	React 18 / Vite 5	Modern, component-based user interface.
Database	PostgreSQL 16	Reliable relational storage with schema-level isolation.
Messaging	RabbitMQ	Durable, asynchronous message brokering.
Monitoring	Prometheus & Grafana	Metrics collection and visual dashboards.
Container	Docker / Compose v2	Environment consistency across Dev/Test/Prod.
Orchestration	Kubernetes (Minikube)	Self-healing, scalable container cluster.

Table 3.1: System Infrastructure Overview

3.2 Database Multi-Tenancy Strategy

While using a single PostgreSQL instance for resource efficiency, we maintain strict logical separation using **four distinct schemas**. This allows us to simulate a "Database-per-Service" pattern without the overhead of multiple database servers.

Chapter 4

Monitoring & Observability

4.1 The Monitoring Stack

To ensure the high availability of FoodieGo, we implemented a full-stack monitoring solution that captures data at the service, container, and node levels.

Prometheus: Acts as our central time-series database. It is configured to "scrape" metrics every 15 seconds from all active microservices and the Kubernetes cluster itself.

Grafana: Provides the visualization layer. We created custom dashboards to monitor API response times, request counts, and error rates (5xx/4xx errors).

cAdvisor: Integrates with Prometheus to provide real-time resource usage statistics (CPU usage, Memory saturation, and Network throughput) for every Docker container.

4.2 Health Monitoring

Each microservice exposes a `/health` endpoint. Kubernetes uses these for **Liveness** and **Readiness** probes, ensuring that traffic is only routed to healthy pods and that failing services are automatically restarted.

Chapter 5

Setup and Deployment

5.1 Environment Configuration

The system is optimized for **Ubuntu 22.04**. Before deployment, copy the environment template:

```
1 cp .env.example .env
2 # Fill in your secrets and RabbitMQ credentials
```

5.2 Kubernetes Pipeline

For a production-like simulation, use the following Minikube workflow:

```
1 # Point Docker CLI to Minikube registry
2 eval $(minikube docker-env)
3
4 # Build backend images
5 docker build -t food-delivery/user-service ./services/user-service
6 docker build -t food-delivery/order-service ./services/order-service
7
8 # Deploy all K8s manifests (Deployments, Services, ConfigMaps)
9 kubectl apply -f k8s/
10
11 # Launch the monitoring and frontend services
12 minikube service grafana-service
13 minikube service frontend-service
```

Chapter 6

Closing Remarks

FoodieGo demonstrates the practical application of advanced cloud computing concepts. By transitioning from a monolithic approach to a microservices architecture, and incorporating robust monitoring via **Prometheus** and **Grafana**, we have built a system that is not only scalable but also resilient and observable.